

תכנון טכני מפורט

מבנה התיקיות בפרויקט

מאורגן לפי **domain**, לא לפי שכבה טכנית — לפי העיקרון ב- **CLAUDE.md** : הוספת פיצ'ר משמעה הוספת מודול, לא עריכת כמה קבצים משותפים. החלטה זו התקבלה כדי להקל על ביצוע שינויים לאפליקציה בהמשך.

```
/
├── app/
│   ├── (auth)/login/page.tsx
│   ├── (admin)/
│   │   ├── layout.tsx # ניווט משותף + admin-only שומר
│   │   └── admin/
│   │       ├── qualifications/page.tsx
│   │       ├── positions/page.tsx
│   │       ├── personnel/page.tsx + [workerId]/page.tsx
│   │       ├── shift-templates/page.tsx
│   │       ├── shifts/page.tsx + past/page.tsx
│   │       ├── availability-windows/page.tsx + [id]/page.tsx
│   │       ├── schedule/[windowId]/page.tsx
│   │       └── settings/page.tsx # עמוד הגדרות מאוחד - אילוצי שיבוץ,
│   └── העדפות זיווג,
│       └── הגדרות אפליקציה כלליות
│   ├── (worker)/
│   │   ├── layout.tsx # worker-only שומר
│   │   ├── my-qualifications/page.tsx
│   │   ├── availability/page.tsx
│   │   ├── my-shifts/page.tsx
│   │   └── notifications/page.tsx
│   ├── dashboard/page.tsx # מסך בית מודע-תפקיד
│   └── error.tsx # error boundary (ראו - נוסף מאוחר יותר)
├── ("טיפול בשגיאות")
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── ui/ # שנוצרו (14 קבצים) shadcn/ui רכיבי
│   └── shared/ # AppHeader.tsx, nav-links.ts
├── features/ # domain תיקייה אחת לכל
│   ├── auth/ { actions.ts (login/logout),
components/LoginForm.tsx }
│   ├── accounts/ { actions.ts, queries.ts, schema.ts, components/ }
│   ├── qualifications/ { actions.ts, queries.ts, schema.ts, components/ }
│   ├── positions/ { actions.ts, queries.ts, components/ }
│   ├── worker-qualifications/ { actions.ts, queries.ts, components/ }
│   ├── shift-templates/ { actions.ts, queries.ts, components/ }
│   └── shifts/ { actions.ts, queries.ts, schema.ts, status.ts,
```

```

components/ }
|   |— availability-windows/{ actions.ts, queries.ts, schema.ts, components/ }
|   |— availability/         { actions.ts, queries.ts, components/ }
|   |— scheduling/           { heuristic.ts (פונקציה טהורה), actions.ts,
queries.ts, components/ }
|   |— settings/             { actions.ts, queries.ts, components/ } — נפרד מ-
scheduling/ במכוון
|   |— notifications/        { actions.ts, queries.ts, components/ }
|   |— dashboard/            { components/ AdminDashboard/WorkerDashboard,
מרכיבות
|
|                               האחרים, לא בעלות domains-שאילות מה
} שאילות/פעולות משלה
|— lib/
|   |— supabase/ { server.ts, client.ts, proxy.ts, admin.ts }
|   |— auth.ts    # requireAdmin(), requireWorker()
|   |— result.ts  # משותף Result<T> טיפוס
|   |— utils.ts
|— supabase/
|   |— migrations/ # גרסאות 9
|   |— seed.sql    # נתוני פיתוח סינתטיים
|— types/database.types.ts # נוצר via `supabase gen types`
|— scripts/create-admin.mjs # בוטסטרפף לחשבון מנהל ראשון
|— tests/
|   |— integration/ # Server Actions אמיתי Supabase מול פרויקט
|   |— setup-rtl.ts # React Testing ניקוי + jest-dom של matchers
Library
|— proxy.ts # בכל בקשה session-מרענן את ה
|— docs/    # מסמכי הקורס

```

כל קובץ `features/<domain>/actions.ts` הוא המקום היחיד שכותב לטבלאות ה-domain שלו — עמודים קוראים לו, אף פעם לא שואלים/כותבים ל-Supabase ישירות עבור נתונים של domain אחר.

מבנה הקומפוננטות המרכזיות

- ה- `components/shared/AppHeader.tsx` (**Layout/shell:** `<AppHeader>`) — כותרת דביקה משותפת (שם האפליקציה + תפריט hamburger ב- `Sheet`), משמשת גם את `(admin)` וגם את `(worker)`, כל אחד מעביר את רשימת ה- `links` שלו (`components/shared/nav-links.ts`).
- **תצוגת סטטוס:** `features/shifts/components/` `<ShiftStatusBadge>`, מבוססת על `features/shifts/status.ts`'s `getShiftStatus` / `isShiftEditable` — טיטה/שובצה/פורסמה, בשימוש גם ברשימה וגם בתצוגת היום. סטטוס כשירות (מאושר/ממתין/עומד לפוג/פג) מוצג בתוך `WorkerQualificationsList` / `MyQualificationsList`, לא רכיב `badge` נפרד.
- **דשבורדים:** `features/dashboard/components/` `<AdminDashboard>` / `<WorkerDashboard>` — קומפוזיציה בלבד: כל `widget` קורא לשאילתה של ה-domain שבאמת מחזיק אותה (`features/shifts/queries.ts` ב- `listUnderstaffedShifts` / `listUpcomingShifts` ב- `features/worker--` `listPendingApprovals` / `listExpiringQualifications` ב- `features/worker--` `qualifications/queries.ts` וכו') — לא לדמיין דשבורד משלו.

- **טפסים:** `<PositionForm>`, `<QualificationForm>` (כולל `QualificationMultiPicker` לבחירת כשירויות נדרשות/מתחדשות), `<ShiftForm>` — משמש גם ל"יצירה מאפס" וגם ל"יצירה מתבנית" (בחירת תבנית רק ממלאת מראש את אותם ערכי טופס).
- ה-`(client) <ShiftsPanel>` — הרכיב שמחזיק את מצב תצוגת-רשימה/יומן ומשמרת נבחרת עבור `<ShiftsCalendar>`; `/admin/shifts` (תצוגות חודש/שבוע) ו-`<ShiftReadOnlyView>` (למשמרת שכבר פורסמה) בנויים סביבו.
- ה-`<AvailabilityShiftRow>` — שורת המשמרת בעמוד `/availability` של העובד, עם `toggle` אופטימי (`useOptimistic`).
- ה-`<ScheduleShiftCard>` / `<ScheduleShiftsList>` — מסך בחינה/עריכה של לוח המשמרות (`/admin/schedule/[windowId]`): כרטיסים מתקפלים לכל משמרת, מציגים את שיבוצי המנוע, ניתנים לעריכה לפני פרסום. הרכיב היחיד שמכוון בכוונה לדסקטופ (ראו `architecture.md` → עיצוב רספונסיבי).

מבנה בסיס הנתונים (ברמת עמודה)

טיפוסים הם טיפוס `Postgres`; כל הטבלאות משתמשות במפתח ראשי `uuid` (`default` `gen_random_uuid()` אלא אם צוין אחרת. מרענן את רשימת הישיות של `architecture.md` עם עמודות ואילוצים מדויקים.

```
profiles
  id          uuid PK -- = auth.users.id
  full_name   text      not null
  role        text      not null check (role in ('admin','worker'))
  created_at  timestampz not null default now()

qualifications
  id          uuid PK
  name        text      not null unique
  renewal_interval_days integer null -- לא פג לעולם = null
  created_at  timestampz not null default now()

qualification_options
  id          uuid PK
  qualification_id uuid    not null references qualifications(id) on delete cascade
  label       text      not null
  sort_order  integer    not null default 0
  created_at  timestampz not null default now()
  unique (qualification_id, label)

positions
  id          uuid PK
  name        text      not null unique
  created_at  timestampz not null default now()

position_qualifications
  position_id uuid references positions(id) on delete cascade -- כשירויות נדרשות לכל תפקיד
```

```

qualification_id  uuid references qualifications(id) on delete cascade
option_id        uuid null references qualification_options(id) on delete
restrict
primary key (position_id, qualification_id)
-- דרישה יכולה למקד ערך ספציפי (למשל "דורש דרגה = סר"ן"), נאכף על ידי טריגר
(נדרש אם ורק אם)
-- לכשירות יש אפשרויות, וחייב להשתייך לאותה כשירות). לא נוסף בכוונה ל
position_renews_qualifications
-- חידוש מאריך תוקף לאיזו אפשרות שהעובד כבר מחזיק, לא ממקד אחת (זה תהליך -
.קידום, לא חידוש)

position_renews_qualifications  -- many-to-many כשירויות שתפקיד מחדש
position_id  uuid references positions(id) on delete cascade
qualification_id  uuid references qualifications(id) on delete cascade
primary key (position_id, qualification_id)

worker_qualifications
id  uuid PK
worker_id  uuid references profiles(id) on delete cascade
qualification_id  uuid references qualifications(id) on delete cascade
option_id  uuid null references qualification_options(id) on
delete restrict
source  text  not null check (source in
('self_reported','admin_granted'))
status  text  not null default 'pending' check (status in
('pending','approved','rejected'))
obtained_at  date  not null
reviewed_by  uuid  references profiles(id) null
reviewed_at  timestampz null
created_at  timestampz not null default now()
-- (worker, rejected אינדקס ייחודי חלקי: לכל היותר רשומה אחת שאינה
qualification) -
-- כפולות, עדיין מאפשר דיווח חוזר אחרי דחייה pending/approved מונע שורות
unique (worker_id, qualification_id) where (status <> 'rejected')

shift_templates
id  uuid PK
name  text  not null
created_at  timestampz not null default now()

shift_template_positions
template_id  uuid references shift_templates(id) on delete cascade
position_id  uuid references positions(id) on delete restrict
headcount_needed  integer not null check (headcount_needed > 0)
primary key (template_id, position_id)

availability_windows
id  uuid PK
label  text  not null

```

```
opens_at      timestampz not null
closes_at     timestampz not null check (closes_at > opens_at)
created_at    timestampz not null default now()
```

shifts

```
id            uuid PK
date          date      not null
start_time    time       not null
end_time      time       not null check (end_time > start_time)
location      text       null
name          text       null -- 2026-08- נוסף, שם תצוגה אופציונלי
```

27

```
availability_window_id  uuid      references availability_windows(id) null
published_at            timestampz null
created_at              timestampz not null default now()
```

shift_positions

```
shift_id      uuid references shifts(id) on delete cascade
position_id   uuid references positions(id) on delete restrict
headcount_needed integer not null check (headcount_needed > 0)
primary key (shift_id, position_id)
```

availability

```
id            uuid PK
worker_id     uuid references profiles(id) on delete cascade
shift_id      uuid references shifts(id) on delete cascade
is_available  boolean      not null
responded_at  timestampz not null default now()
unique (worker_id, shift_id) -- תגובה אחת לעובד למשמרת; הגשה חוזרת = עדכון
```

assignments

```
shift_id      uuid references shifts(id) on delete cascade,
position_id   uuid references positions(id) on delete restrict,
worker_id     uuid references profiles(id) on delete cascade,
created_by    uuid references profiles(id) null, -- null = נוצר על ידי המנוע;
```

מוגדר = מנהל

```
created_at    timestampz not null default now(),
primary key (shift_id, position_id, worker_id),
foreign key (shift_id, position_id) references shift_positions(shift_id, position_id)
```

notifications

```
id            uuid PK
worker_id     uuid references profiles(id) on delete cascade
shift_id      uuid references shifts(id) on delete set null null
message       text       not null
created_at    timestampz not null default now()
read_at       timestampz null
```

```

scheduling_constraints -- פרמטרים הניתנים לכיוון על ידי המנהל, נקראים
בשלב הזכאות
  id                uuid PK
  type              text      not null check (type in ('min_rest_hours',
'max_shifts_per_window'))
  qualification_option_id uuid  null references qualification_options(id)
on delete cascade
  -- חריגה לעובדים = null-שורת ברירת המחדל/הגלובלית של הסוג. לא = null
המחזיקים באותה אפשרות
  enabled           boolean    not null default false
  value             numeric     not null -- או שעות, או
מונה
  updated_at        timestampz not null default now()
  unique index on (type) where qualification_option_id is null
  unique index on (type, qualification_option_id) where qualification_option_id
is not null

worker_pairing_preferences -- יחסי, לא אילוף זכאות פר-עובד - ראו "מנוע
השיבוץ" בהמשך
  id                uuid PK
  worker_id_1       uuid       references profiles(id) on delete cascade
  worker_id_2       uuid       references profiles(id) on delete cascade
  preference         text       not null check (preference in ('avoid',
'prefer_avoid', 'prefer'))
  created_at        timestampz not null default now()
  check (worker_id_1 < worker_id_2) -- סדר קנוני: שורה אחת לזוג, לא שתיים
  unique (worker_id_1, worker_id_2) -- זוג מחזיק העדפה אחת בדיוק בכל רגע
נתון

availability_change_requests -- נוסף 2026-08-28: בקשת עובד לשינוי אחרי סגירת
חלון זמינות
  id                uuid PK
  worker_id         uuid       references profiles(id) on delete cascade
  shift_id          uuid       references shifts(id) on delete cascade
  message           text       null
  created_at        timestampz not null default now()
  acknowledged_at   timestampz null
  acknowledged_by   uuid       references profiles(id) null

app_settings -- singleton, 2026-08-27 נוסף
  id                uuid PK
  expiring_soon_days integer    not null default 30
  updated_at        timestampz not null default now()
  unique index on ((true)) -- לכל היותר שורה אחת, אי פעם

```

פעולות CREATE/READ/UPDATE/DELETE מרכזיות

ישות	יצירה	קריאה	עדכון	מחיקה
qualifications	מנהל	כולם	מנהל	מנהל (חסום אם מוזכר במקום אחר)
positions	מנהל	כולם	מנהל	מנהל (חסום אם מוזכר)
worker_qualifications	עובד (עצמי, → pending) / מנהל (כל עובד, → approved)	עצמי (עובד) / הכול (מנהל)	מנהל בלבד (אישור/דחייה/ביטול)	— (נשמר כהיסטוריה; ביטול הוא עדכון סטטוס, לא מחיקה)
shift_templates	מנהל	מנהל	מנהל	מנהל
shifts	מנהל (ריק או מתבנית)	כולם (עובדים — רק רלוונטי)	מנהל	מנהל (רק אם לא פורסמה)
availability_windows	מנהל	כולם	מנהל (תאריכים)	מנהל
availability	עובד (עצמי)	עצמי (עובד) / הכול (מנהל)	עובד (עצמי, לפני סגירת החלון)	— (הגשה חוזרת דורסת, אין מחיקה נפרדת)
assignments	מנוע (בכמות, דרך generate) / מנהל (ידני)	מנהל (הכול) / עובד (עצמי, רק מפורסם)	מנהל (שיבוץ מחדש, לפני פרסום)	מנהל (לפני פרסום)
notifications	מערכת (בפרסום)	עצמי (עובד)	עובד (סימון כנקרא)	—
scheduling_constraints	— (שורות ברירת מחדל נזרעות במיגרציה; שורות חריגה נוצרות על ידי מנהל)	מנהל	מנהל (הפעלה + ערך)	מנהל (שורות חריגה בלבד)
worker_pairing_preferences	מנהל	מנהל	מנהל (שינוי סוג ההעדפה)	מנהל
availability_change_requests	עובד (עצמי)	עצמי (עובד) / מנהל (הכול)	מנהל (סימון כנצפה)	—
app_settings	— (שורה יחידה נזרעת במיגרציה)	כולם	מנהל	—

שני "לא-מוחקים" מכוונים ששווה להדגיש: **היסטוריית כשירות** של עובד אף פעם לא נמחקת פיזית (ביטול הוא שינוי סטטוס, כך ש"מי החזיק מה ומתי" נשאר ניתן לשאלתה), ו- `qualifications / positions` חוסמות מחיקה ברגע שהן מוזכרות במקום אחר במקום להשתלשל (`delete cascade`) — מחיקת כשירות ש-50 עובדים מחזיקים, או תפקיד המשמש בעשרות משמרות, צריכה להיות החלטה מכוונת שהממשק מציג בבירור, לא מפולת שקטה.

תיאור ה-API

החתימות בפועל (לא "מייצגות" כמו בטיפוטה המקורית — זה מה שבאמת נבנה ונבדק):

```
// features/auth/actions.ts
login(prevState, formData: { email, password }): { error: string } | undefined
// בהצלחה -/dashboard מפנה ל
logout(): void // -/login מפנה ל

// features/accounts/actions.ts + queries.ts
createWorkerAccount(prevState, formData: { fullName, email, password }):
Result<void> // מנהל בלבד
listWorkers(): Promise<Profile[]>
getWorker(id): Promise<Profile | null>

// features/qualifications/actions.ts + queries.ts
createQualification(prevState, formData): Result<Qualification>
updateQualification(id, prevState, formData): Result<Qualification>
deleteQualification(id): Result<void> // נכשל אם עדיין מוזכר
listQualifications(): Promise<Qualification[]>

// features/positions/actions.ts + queries.ts
createPosition(prevState, formData): Result<{ id: string; name: string }> //
מחזיר את שנוצר, לצורך

// מתוך טופס משמרת inline יצירה
updatePosition(id, prevState, formData): Result<Position>
deletePosition(id): Result<void>
listPositions(): Promise<Position[]>

// features/worker-qualifications/actions.ts + queries.ts
grantQualification(prevState, formData): Result<WorkerQualification> //
מנהל בלבד
selfReportQualification(prevState, formData): Result<WorkerQualification> //
עובד, → pending
revokeQualification(id, workerId): Result<void> //
מנהל בלבד
reviewQualification(id, decision, workerId): Result<void> //
מנהל בלבד, אישור/דחייה
computeExpiresOn(...): Date | null // נבדקת יחידה בנפרד
listWorkerQualifications(workerId): Promise<WorkerQualification[]>
listPendingApprovals(limit): Promise<PendingApproval[]>
```

```

listExpiringQualifications(...): Promise<...>

// features/shift-templates/actions.ts + queries.ts
createShiftTemplate(prevState, formData): Result<ShiftTemplate>
updateShiftTemplate(id, prevState, formData): Result<ShiftTemplate>
deleteShiftTemplate(id): Result<void>
listShiftTemplates(): Promise<ShiftTemplate[]>

// features/shifts/actions.ts + queries.ts
createShift(prevState, formData): Result<Shift> // fromTemplateId
updateShift(id, prevState, formData): Result<Shift>
deleteShift(id): Result<void> // נכשל אם פורסמה
listShifts({ scope: "all" | "upcoming" | "past" }): Promise<ShiftListItem[]>
listUpcomingShifts(limit): Promise<UpcomingShift[]>
listUnderstaffedShifts(limit): Promise<UnderstaffedShift[]>
listMyUpcomingShifts(workerId): Promise<MyShift[]>

// features/availability-windows/actions.ts + queries.ts
createAvailabilityWindow(prevState, formData): Result<AvailabilityWindow>
updateAvailabilityWindow(id, prevState, formData): Result<AvailabilityWindow>
deleteAvailabilityWindow(id): Result<void>
listAvailabilityWindows(): Promise<AvailabilityWindow[]>
listWindowsPendingApproval(limit): Promise<PendingApprovalWindow[]>
getAvailabilityWindowDetail(...): Promise<...>

// features/availability/actions.ts + queries.ts
submitAvailability(prevState, formData: { shiftIds: string[] }): Result<void>
// upsert לפי (worker_id, shift_id)
requestAvailabilityChange(prevState, formData): Result<void> // אחרי סגירת
חלון
acknowledgeChangeRequest(id, windowId): Result<void> // מנהל בלבד
listOpenWindowsWithShifts(workerId): Promise<OpenWindow[]>
listRecentlyClosedWindowsWithResponses(...): Promise<...>
listChangeRequestsForWindow(windowId): Promise<AdminChangeRequest[]>
listPendingChangeRequests(limit): Promise<AdminChangeRequest[]>

// features/scheduling/heuristic.ts – DB פונקציה טהורה, ללא
runSchedulingHeuristic(input: HeuristicInput): { assignments, unfilledSlots,
softAvoidConflicts }

// features/scheduling/actions.ts + queries.ts
generateSchedule(windowId): Result<{ proposedCount; unfilledSlots;
softAvoidConflicts }>
addAssignment(windowId, shiftId, positionId, workerId): Result<void> // שינוי
ידני של המנהל
removeAssignment(windowId, shiftId, positionId, workerId): Result<void>
publishShift(windowId, shiftId): Result<void> // פר-משמרת, לא כל החלון
בבת אחת
unpublishShift(windowId, shiftId): Result<void>

```

```

publishAllShiftsInWindow(windowId): Result<{ publishedCount }>
setSchedulingConstraint(type, qualificationOptionId, prevState, formData):
Result<void>
deleteSchedulingConstraintOverride(id): Result<void>
setWorkerPairingPreference(prevState, formData): Result<void> // upsert לפי
הזוג הקנוני
deleteWorkerPairingPreference(id): Result<void>
listSchedulingConstraints(): Promise<ConstraintRow[]>
listWorkerPairingPreferences(): Promise<WorkerPairing[]>
getScheduleReview(windowId): Promise<ScheduleReview | null>
listActivePairingConflicts(limit): Promise<ActivePairingConflict[]>

// features/settings/actions.ts + queries.ts – scheduling/ במכוון:
expiring_soon_days
// מרכיב את /admin/settings לא שיבוץ – עמוד worker-qualifications הוא עניין של
שניהם, זו
// אגרגציה בממשק, לא מיזוג בשכבת הנתונים
updateExpiringSoonDays(prevState, formData): Result<void> // מנהל בלבד
getAppSettings(): Promise<AppSettings>

// features/notifications/actions.ts + queries.ts
markNotificationRead(id): Result<void>
markAllNotificationsRead(): Result<void>
listMyNotifications(workerId): Promise<Notification[]>
countUnreadNotifications(workerId): Promise<number>

```

תיאור הלוגיקה העסקית המרכזית

1. תוקף כשירות – מחושב, לא נשמר

עבור (worker, qualification) נתון:

```

last_renewed_on = max(
  worker_qualifications.obtained_at,
  max(shift.date)
  :על כל המשמרות שהושלמו כאשר
  - העובד שובץ לתפקיד באותה משמרת
  - עבור כשירות זו position_renews_qualifications-אותו תפקיד מופיע ב
  - shift.date <= היום ("לפי ההנחה ב" product-spec.md-הושלמה",
)

expires_on =
  null // הוא renewal_interval_days אם
  null
  last_renewed_on + renewal_interval_days // אחרת

```

N = ימים, N הסטטוס המוצג למשתמשים = "מאושר" / "עומד לפוג בקרוב" (בתוך
expiring_soon_days

"ברירת מחדל 30 – הוחלט וניתן להגדרה על ידי המנהל) / "פג תוקף, app_settings-ב
(expires_on > היום)
לא נשמר, expires_on-מחושב מ –

מומש כפונקציית שאילתה, לא כ-view נשמר – computeExpiresOn / getLatestRenewalDates
ב- features/worker-qualifications/queries.ts, נבדק ביחידה ב- features/worker-
qualifications/__tests__/computeExpiresOn.test.ts (מקרי גבול: אף פעם לא פג, מרווח פשוט,
הארכה על ידי חידוש, שוויון על הגבול, מעבר חודש/שנה).

מגבלה ידועה, אושרה במפורש כלא בהיקף גרסה 1: לחישוב הזה אין מודעות קדימה – הוא מגיב רק לשיבוצים
שכבר בוצעו; ניקוד ההיריסטיקה (בהמשך) אינו מודע לכך שכשירות של עובד עומדת לפוג ולכן לא מעדיף לשבץ אותו
לתפקיד שהיה מחדש אותה. שלב הבדיקה/עריכה הידני של המנהל לפני פרסום הוא הדרך לתקן זאת ידנית היום.

2. מנוע השיבוצ – ההיריסטיקה המתאימה

רץ פעם אחת בכל קריאה ל- generateSchedule(windowId). **ההיריסטיקה חמדנית (greedy)** במכוון, לא
פותר אילוצים מלא (ראו architecture.md) – ניתנת להסבר במעבר אחד, עם בחינה ידנית של המנהל כרשת
ביטחון לנכונות.

שני סוגי קלט הניתנים להגדרה על ידי המנהל מזינים את האלגוריתם, והם מתנהגים אחרת:

- **scheduling_constraints** – כללים **פר-עובד** (האם העובד הזה, לבדו, כשיר). לכל סוג יש ערך ברירת
מחדל וחריגות אופציונליות לפי אפשרות-כשירות. הערך האפקטיבי של עובד לסוג אילוצי = חריגה התואמת
הספציפית ביותר אם קיימת, אחרת שורת ברירת המחדל. אלה **קשיחים** – עובד שנכשל באחד פשוט אינו זכאי
למשבצת.
- **worker_pairing_preferences** – **יחסי** (האם העובד הזה, בשילוב עם מי שכבר במשמרת, יוצר זיווג
טוב/רע). שלושה סוגים, רק אחד מהם קשיח:
 - **avoid** – **קשיח**. עובד לא זכאי למשבצת אם עובד שהוא "avoid" איתו כבר משובץ (בהרצה הזו) לתפקיד
אחר **באותה משמרת**. אם זה משאיר אפס עובדים זכאים, המשבצת נשארת ריקה ומסומנת ככל פער אחר
– לעולם לא נעקפת.
 - **prefer_avoid** – **רך**. לא משפיע על זכאות. רק מוריד עדיפות בשובר השוויון (שלב 4) כאשר שותף
"prefer_avoid" כבר במשמרת – בשימוש רק אם זו הדרך היחידה למלא את המשבצת. כל זיווג שקורה
בכל זאת חייב להיות מסומן למנהל גם לפני וגם אחרי פרסום.
 - **prefer** – **רך**, ההפך: מעלה עדיפות בשובר השוויון כאשר שותף מועדף כבר במשמרת. לעולם לא כופה
זיווג, רק מטה אליו.

זוג מחזיק בדיוק אחד משלושת אלה בכל רגע נתון (נאכף על ידי אילוצי הייחודיות של הטבלה על הזוג).

1. **פירוק למשבצות**. לכל משמרת בחלון, לכל שורת shift_positions, נוצרת משבצת אחת לכל יחידת
headcount_needed.
2. **חישוב זכאות לכל משבצת**. עובד זכאי אם: הוא מחזיק כשירות **מאושרת** המספקת כל כשירות שהתפקיד של
המשבצת דורש (position_qualifications); הוא הגיש is_available = true לאותה משמרת;
שיבוצו לא יחפוף משמרת אחרת שכבר שובצה לו באותו תאריך (מניעת שיבוצ כפול); הוא עומד בכל שורת

`scheduling_constraints` מופעלת בערך האפקטיבי שלו; ואין שותף "avoid" כבר משובץ לתפקיד אחר באותה משמרת, בהרצה הזו. שורות אילוץ מכובות מדולגות לגמרי.

3. **מיון משבצות לפי נדירות.** מיון עולה לפי מספר העובדים הזכאים — מילוי המשבצות הקשות ביותר קודם, כי השארתן לסוף מסתכנת בכך שהעובדים הזכאים המעטים שלהן כבר נצרכו על ידי משבצות קלות יותר.

4. **שיבוץ.** לכל משבצת לפי הסדר הזה, ניקוד כל עובד זכאי: החל ממספר השיבוצים הכי נמוך בהרצה זו (שובר השוויון המקורי של הוגנות — מפזר עומס במקום לבחור תמיד את הזכאי הראשון), ואז התאמה לפי זיווג — העלאה לעובד עם שותף "prefer" כבר במשמרת, הורדה (לא פסילה) לעובד עם שותף "prefer_avoid" כבר במשמרת. שיבוץ העובד המדורג הכי גבוה. אם השיבוץ זיווג "prefer_avoid" בכל זאת (לא הייתה אפשרות טובה יותר), הוא נרשם כקונפליקט זיווג רך למשמרת הזו.

5. **דיווח על פערים.** משבצת עם אפס עובדים זכאים נשארת ריקה ומדווחת (`unfilledSlots`) — זה בדיוק מה שמניע את דגל "משמרת תת-מאוישת" בדשבורד.

6. **כתיבת שיבוצים מוצעים** (`created_by = null`) — שום דבר לא מתפרסם עדיין; המנהל בוחן/עורך דרך `addAssignment / removeAssignment` לפני קריאה לפרסום. קונפליקטים רכים משלב 4 חייבים להיות מסומנים בבירור במסך הבחינה, **ולהישאר ניתנים לשאילתה גם אחרי פרסום** — מחושבים בזמן קריאה מ- `assignments + worker_pairing_preferences`, לא נשמרים כשורה נפרדת, עקבי עם ההטיה של הפרויקט הזה כלפי "מחושב, לא נשמר" במקומות אחרים.

זו במכוון הגרסה הראשונה של האלגוריתם. `max_shifts_per_window` ו- `min_rest_hours` הם סוגי האילוץ ההתחלתיים — גנריים מספיק כדי להיות שימושיים בלי קלט ספציפי לטייסת, וניתנים לכיוון על ידי המנהל ללא שינוי קוד. כללים ספציפיים לטייסת שיזוהו מאוחר יותר יהפכו לסוגי אילוץ **חדשים**: case חדש בשלב 2 + מיגרציה לזריעת השורה — שינוי קטן ותוספתי, לא עיצוב מחדש.

נבדק ביחידה: 18 בדיקות Vitest ב- `features/scheduling/__tests__/heuristic.test.ts`, מכסות כל אחד מהסעיפים למעלה (התאמת כשיריות כולל אפשרות שגויה, זמינות, שיבוץ כפול קיים ובתוך הרצה, כל שלוש רמות `min_rest_hours` כולל `max_shifts_per_window`, `override`, שלושת סוגי הזיווג, שובר השוויון של הוגנות, וסדר לפי נדירות).

ניהול State באפליקציה

ללא ספריית `client state` גלובלית (לא `Redux/Zustand/React Query`). לא בשימוש במכוון: כש-Server Components שולפות נתונים טריים בכל ניווט ו-Server Actions מפעילות `revalidatePath` (`revalidation`) אחרי שינוי, אין `cache` נפרד בצד לקוח לשמור מסונכרן — השרת הוא מקור האמת היחיד בכל בקשה.

מה כן משתמש ב-`state` מקומי של React, והיכן:

- **קלט טפסים** — `useState / inputs` מבוקרים פשוטים בתוך כל רכיב טופס; לא משותף מחוץ לרכיב.
- **מצב ממשק זמני** (מודל פתוח/סגור, טאב פעיל, מיון טבלה) — `useState` מקומי, אף פעם לא מורם גבוה יותר מהנדרש.
- **עדכונים אופטימיים** עבור כמה אינטראקציות שצריכות להיות חדות במיוחד (למשל `toggle` של תגובת זמינות) — `useOptimistic` של React, כך שהממשק מתעדכן מיידית בזמן שה-Server Action מאשר ברקע, עם חזרה אחורה בכישלון.

טיפול בשגיאות

- כל Server Action מחזירה union מבחין משותף במקום לזרוק:

```
type Result<T> = { success: true; data: T } | { success: false; error: string };
```

- **צרכנים** (רכיבי לקוח) מסתעפים לפי `success` ומציגים `error inline` – שום `exception` גולמי לא חוצה את גבול שרת/לקוח, ושום טקסט שגיה פנימי (`stack trace`, שגיות `Postgres` גולמית) לא מגיע לממשק.
- **שגיאות אימות** (`Zod`) ממופות להודעות ברמת שדה, מוצגות ליד הקלט הרלוונטי.
- **שגיאות מסד נתונים** (דחיית `RLS`, הפרת אילוץ ייחודיות/`check`) נתפסות ב-`action` ומתורגמות להודעה בשפה פשוטה.
- **שגיאות בזמן רינדור**: `error boundary` – `app/error.tsx` לכל `route segment`, ממוקם כך ששגיאות תוכן עמוד לא מפילה איתה את ה-`(worker)/(admin)` `layout` שמסביבה (וכך גם את הכותרת המשותפת) – המשתמש רואה מסך שגיה ידידותי במקום קריסה גולמית של האפליקציה.
- **מקרי "לא נמצא"** `id` שגוי בכתובת, למשל `notFound()` → `notFound(): /admin/personnel/<bad-id>` → `found.tsx`.

ולידציות של קלטים

- כל Server Action מאמתת את הקלט שלה עם **סכמת Zod** הממוקמת ב- `schema.ts` של אותו domain (לא בתוך `actions.ts` — ראו "מבנה התיקיות" למעלה למה), לפני נגיעה במסד הנתונים — זו שכבת האימות הסמכותית, מכיוון ש-Server Actions רצות בצד שרת ללא תלות במה שהלקוח בדק או לא בדק.
- אימות בצד לקוח (`required` של HTML, פענוח Zod מידי בטופס) קיים אך ורק לחוויית משתמש רספונסיבית — לעולם לא נסמך עליו כבדיקה האמיתית, מכיוון שניתן לעקוף אותו בקלות.
- כללים מייצגים: `name` של כשירות לא ריק, אורך מקסימלי, "יחודי"; `renewalIntervalDays` מספר שלם חיובי או `endTime > startTime; null; למשמרת; headcountNeeded > 0;` בוליאני.
- **אילוצי מסד הנתונים הם קו ההגנה האחרון** (אילוצי `check / not null / unique` בסכמה למעלה) — למקרה שבאג אי פעם יאפשר לנתונים פגומים להגיע ישירות לשאילתה.

תכנון חויית המשתמש המרכזית

- **דשבורדים כבסיס בית**, לשני התפקידים — מציגים מה דורש תשומת לב (מנהל: משמרות תת-מאוישות, אישורים ממתינים; עובד: משמרות קרובות, כשיריות שעומדות לפוג) במקום נחיתה על עמוד ריק גנרי.
- **תגי סטטוס עקביים** בכל מקום שכשירות או משמרת מופיעה, כך שהמצב מזוהה במבט אחד בלי לקרוא טקסט (צבע: מאושר/ממתין/עומד לפוג/פג; טיוטה/שובצה/פורסמה).
- **רכיב טופס אחד לכל ישות**, בשימוש חוזר ליצירה ולעריכה (ועבור משמרות — גם ליצירה מאפס וגם מתבנית) — מקום אחד לכוון נכון את חווית המשתמש, לא כמה טפסים כמעט-זהים שנסחפים זה מזה עם הזמן.
- **רפונסיבי לפי architecture.md**: עמודי עובד mobile-first, רשת בחינת-לוח-המשמרות של המנהל מותאמת דסקטופ אך עדיין שמישה במובייל.

- **עברית (RTL) לכל אורכה:** `Noto_Sans_Hebrew`, `<html lang="he" dir="rtl">` כפונט האפליקציה, רכיבי `shadcn/ui` נוצרים עם `--rtl` (`utilities` מרווח לוגיים). כל טקסט הממשק כתוב בעברית ישירות – אין מערכת `i18n/translation-key`, מכיוון שזו אפליקציה חד-לשונית. **כלל שחוזר בפועל:** כל ערך תאריך/שעה, קלט או מוצג, מקבל `dir="ltr"` – אחרת אלגוריתם ה-`bidi` של יוניקוד הופך חזותית מקטעים מספריים (`12:30` מוצג כ- `30:12`) למרות שהטקסט הבסיסי תקין. נתפס בפועל ב- `/admin/shifts` וב- `worker-qualifications` ומתועד כדי לחזור ולהישנות (חלונות זמינות, תצוגת המשמרות של העובד, `widget` המשמרות הקרובות בדשבורד) – טבוע במיזוג נתונים מפורמטים ב-LTR לתוך זרימת RTL, לא באג חד-פעמי.